NAME: TANKISO MASOEBE

OPERATING SYSTEM (ADVANCED)WK11

1. 1. RTOS vs. GPOS Comparison Table

A. Detailed Comparison

Category	General-Purpose Operating System (GPOS)	Real-Time Operating System (RTOS)
Primary Goal	Provides a convenient environment for running many applications and managing system resources.	Ensures tasks are completed within specified time limits.
Scheduling Policy	Uses scheduling policies that balance fairness, responsiveness, and overall system performance.	Uses real-time scheduling policies, such as priority-based preemptive scheduling, to meet timing deadlines.
Kernel Size	Generally larger because it supports many features, device drivers, and services.	Often smaller, with fewer services and a focus on predictable execution.
Interrupt Latency	Can vary depending on system load, drivers, and kernel operations.	Designed to provide low and predictable interrupt latency.
System Overhead	Usually higher because of background services, memory management, and other operating system functions.	Often lower, helping tasks execute with predictable timing.
Response Time	Usually fast but not always guaranteed to meet strict deadlines.	Designed to provide predictable response times and meet defined deadlines.
Memory Management	Commonly supports virtual memory, paging, and complex memory management.	May use simpler memory management to reduce unpredictable delays.
Reliability Requirements	Designed for general reliability, usability, and performance across many workloads.	Often designed for dependable operation in systems with strict timing requirements.
Typical Examples	Microsoft Windows, desktop Linux, and macOS.	FreeRTOS, VxWorks, and QNX.
Typical Applications	Personal computers, office software, web browsing, and gaming.	Airbag controllers, industrial automation, medical devices, and robotics.

B. 200-Word Summary: Why a Tuned Linux Kernel Is Not Enough for an Airbag Controller

A General-Purpose Operating System (GPOS), such as Linux, is designed to support multiple applications while balancing performance, fairness, and resource usage. Even when heavily optimized, Linux may experience unpredictable delays caused by scheduling, interrupt handling, device drivers, and other kernel activities.

An airbag controller is a hard real-time system because it must respond to a crash within a strict time limit. If the controller fails to activate the airbag before the deadline, the consequences could be life-threatening. Therefore, the system must provide predictable timing and reliable responses to critical events.

Although a real-time Linux kernel, such as PREEMPT_RT, can significantly improve scheduling responsiveness and reduce latency, optimization alone does not automatically guarantee that every task will meet its deadline under all operating conditions. Meeting hard real-time requirements requires careful analysis, testing, and appropriate hardware and software design.

A dedicated RTOS, such as VxWorks or a suitable safety-certified real-time platform, can provide deterministic scheduling, controlled interrupt latency, and mechanisms for managing high-priority tasks. These features make it more suitable for applications with strict timing requirements.

In conclusion, a heavily tuned Linux kernel may be useful for many real-time applications, but an airbag controller requires verified worst-case response times and rigorous safety validation. The operating system must be selected according to the system's timing and safety requirements.

2. FreeRTOS Simulation Report

1. Introduction

FreeRTOS is a real-time operating system used in embedded systems to manage multiple tasks. It allows tasks to run according to their priorities and helps control when tasks execute. In this practical exercise, two tasks with different priorities are created to print messages to the console. The vTaskDelay() function and semaphores are used to observe how tasks are scheduled and synchronized.

2. Objectives

- To create two tasks with different priorities in FreeRTOS.

- To understand how vTaskDelay() affects task execution.
- To use a semaphore to synchronize two tasks.
- To observe how task priorities affect execution.

3. Simulation Tool

The simulation can be performed using the Wokwi online simulator with an ESP32 board and FreeRTOS.

Programming language: C++

4. Source Code

```
#include <Arduino.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/semphr.h>

SemaphoreHandle_t mySemaphore;

void Task1(void *parameter) {
  while (1) {
    Serial.println("Task 1: High Priority");

    xSemaphoreGive(mySemaphore);

    vTaskDelay(pdMS_TO_TICKS(1000));
  }
}

void Task2(void *parameter) {
  while (1) {
    if (xSemaphoreTake(mySemaphore, portMAX_DELAY)) {
      Serial.println("Task 2: Low Priority");
    }

    vTaskDelay(pdMS_TO_TICKS(1500));
  }
}

void setup() {
  Serial.begin(115200);

  mySemaphore = xSemaphoreCreateBinary();

  if (mySemaphore != NULL) {
    xTaskCreate(Task1, "Task1", 2048, NULL, 2, NULL);
    xTaskCreate(Task2, "Task2", 2048, NULL, 1, NULL);
  }
}
```

```
void loop() {
}
```

5. Explanation of the Code

- Task 1: This is the high-priority task. It prints a message and releases a semaphore every second.
- Task 2: This is the low-priority task. It waits for the semaphore before printing its message.
- vTaskDelay(): Pauses a task for a specified period, allowing other ready tasks to execute.
- Semaphore: Used to coordinate the two tasks. Task 2 waits until Task 1 signals it.
- xTaskCreate(): Creates each task and assigns its priority. Task 1 has priority 2, while Task 2 has priority 1.

6. Observations

Experiment	Observation
Different priorities	The high-priority task is given preference when both tasks are ready.
Using vTaskDelay()	Tasks pause temporarily, allowing other tasks to execute.
Using a semaphore	Task 2 waits until Task 1 releases the semaphore.
Removing the delay	A continuously ready high-priority task can prevent a lower-priority task from running.

Expected Console Output

```
Task 1: High Priority
Task 2: Low Priority
Task 1: High Priority
Task 2: Low Priority
Task 1: High Priority
Task 2: Low Priority
```

Note: This is an example of expected output. The actual output and timing may vary during simulation.

7. Conclusion

In this practical exercise, I learned how FreeRTOS manages multiple tasks using task priorities, delays, and semaphores. I observed that a high-priority task is given preference when it is ready to run. The vTaskDelay() function allows a task to pause

temporarily, while a semaphore enables communication and synchronization between tasks. This practical helped me understand how FreeRTOS can be used to manage tasks efficiently in embedded systems.